

# PLSQL Exercises

## Exercise 3: Stored Procedures

Experiment 2.sql

7

CREATE TABLE employees (

9

10

11

12

13

14

15

16

17

18

19

20

21

name

department

salary

);

INSERT INTO savings\_accounts VALUES (201, 1, 10000);

INSERT INTO savings\_accounts VALUES (202, 2, 7000);

INSERT INTO employees VALUES (301, 'David', 'Sales', 50000);

INSERT INTO employees VALUES (302, 'Eve', 'Sales', 55000);

INSERT INTO employees VALUES (303, 'Frank', 'IT', 60000);

COMMIT;

Query result

Script output

DBMS output

Explain Plan

SQL history

SQL> COMMIT

Commit complete.

Elapsed: 00:00:00.002

**Scenario 1:** The bank needs to process monthly interest for all savings accounts.

Scenario 1 - ProcessMonthlyInterest.sql

Aa

```
9
10 CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
11     ... p_department IN VARCHAR2,
12     ... p_bonus_pct IN NUMBER
13 ) IS
14 BEGIN
15     ... UPDATE employees
16     ... SET salary = salary + (salary * p_bonus_pct / 100)
17     ... WHERE department = p_department;
18
19     ... COMMIT;
20     ... DBMS_OUTPUT.PUT_LINE('Bonus of ' || p_bonus_pct || '% applied to department: ' || p_department);
21 END UpdateEmployeeBonus;
22
```

Query result

Script output

DBMS output

Explain Plan

SQL history

Show more...

Procedure UPDATEEMPLOYEEBONUS compiled

Elapsed: 00:00:00.016

Query result

Script output

DBMS output

Explain Plan

SQL history

Monthly interest applied to all savings accounts.

**Scenario 2:** The bank wants to implement a bonus scheme for employees based on their performance.

Scenario 2 - UpdateEmployeeBonus.sql

Aa

```
1  SET SERVEROUTPUT ON;
2
3  BEGIN
4      ProcessMonthlyInterest;
5  END;
6  /
7
8  BEGIN
9      UpdateEmployeeBonus('Sales', 10);
10 END;
11 /
12
13 SELECT * FROM savings_accounts;
14 SELECT * FROM employees;
```

Query result

Script output

DBMS output

Explain Plan

SQL history

SQL> SELECT \* FROM employees

| EMPLOYEE_ID | NAME  | DEPARTMENT | SALARY |
|-------------|-------|------------|--------|
| 301         | David | Sales      | 55000  |
| 302         | Eve   | Sales      | 60500  |
| 303         | Frank | IT         | 60000  |

Elapsed: 00:00:00.007  
3 rows selected.

Query result

Script output

DBMS output

Explain Plan

SQL history

Bonus of 10% applied to department: Sales

### Scenario 3: Customers should be able to transfer funds between their accounts

Query:

```
CREATE OR REPLACE PROCEDURE TransferFunds (  
    p_from_account  IN NUMBER,  
    p_to_account    IN NUMBER,  
    p_amount        IN NUMBER  
) IS  
    v_balance NUMBER;  
BEGIN  
    SELECT balance INTO v_balance  
    FROM savings_accounts  
    WHERE account_id = p_from_account;  
  
    IF v_balance >= p_amount THEN  
        UPDATE savings_accounts  
        SET balance = balance - p_amount  
        WHERE account_id = p_from_account;  
  
        UPDATE savings_accounts  
        SET balance = balance + p_amount  
        WHERE account_id = p_to_account;  
  
        COMMIT;  
        DBMS_OUTPUT.PUT_LINE('Transfer successful: ' || p_amount ||  
                             ' moved from account ' || p_from_account ||  
                             ' to account ' || p_to_account);  
    ELSE  
        DBMS_OUTPUT.PUT_LINE('Transfer failed: Insufficient balance in account ' ||  
p_from_account);  
    END IF;  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        DBMS_OUTPUT.PUT_LINE('Error: Source account not found.');
```

END TransferFunds;  
/  
  
SET SERVEROUTPUT ON;

-- Test 1: Successful transfer  
BEGIN  
 TransferFunds(201, 202, 1000);  
END;  
/  
  
-- Test 2: Insufficient balance

```

BEGIN
    TransferFunds(201, 202, 999999);
END;
/

-- Test 3: Invalid source account
BEGIN
    TransferFunds(9999, 202, 100);
END;
/

-- Verify final balances
SELECT * FROM savings_accounts;

```

Output:

Experiment - 3 Transfer funds.sql

```

1 CREATE OR REPLACE PROCEDURE TransferFunds (
2     p_from_account IN NUMBER,
3     p_to_account IN NUMBER,
4     p_amount IN NUMBER
5 ) IS
6     v_balance NUMBER;
7 BEGIN
8     SELECT balance INTO v_balance
9     FROM savings_accounts
10    WHERE account_id = p_from_account;
11
12    IF v_balance >= p_amount THEN
13        UPDATE savings_accounts
14        SET balance = balance - p_amount
15        WHERE account_id = p_from_account;
16
17        UPDATE savings_accounts
18        SET balance = balance + p_amount
19        WHERE account_id = p_to_account;

```

Query result
Script output
DBMS output
Explain Plan
SQL history

SQL> BEGIN  
TransferFunds(201, 202, 1000);  
END;

Transfer successful: 1000 moved from account 201 to account 202

PL/SQL procedure successfully completed.

| Query result                                                                      | Script output                                                                     | DBMS output | Explain Plan | SQL history                                                                                                                                                                |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |  |             |              |                                                                                                                                                                            |
| Transfer successful: 1000 moved from account 201 to account 202                   |                                                                                   |             |              | <br> |
| Transfer failed: Insufficient balance in account 201                              |                                                                                   |             |              | <br> |
| Error: Source account not found.                                                  |                                                                                   |             |              | <br> |

Final Query Result:

| Query result                                                                      | Script output                                                                     | DBMS output | Explain Plan                  | SQL history |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------|-------------------------------|-------------|
|  |  | Download ▾  | Execution time: 0.002 seconds |             |
|                                                                                   | ACCOUNT_ID                                                                        | CUSTOMER_ID | BALANCE                       |             |
| 1                                                                                 | 201                                                                               | 1           | 9100                          |             |
| 2                                                                                 | 202                                                                               | 2           | 8070                          |             |